

4 Programming Languages

Hui Xu, xuh@fudan.edu.cn

Learning Objective:

- Understand the concept of programming languages.
- Learn to read programs written in Python.
- Practice using AI coding agents to write Python programs.

4.1 Overview of Programming Languages

A programming language is a formal system for specifying computation. It provides a way for humans to write instructions that a computer can understand and execute. According to Kenneth Iverson in his book *A Programming Language* (1962):

“Applied mathematics is largely concerned with the design and analysis of explicit procedures for calculating the exact or approximate values of various functions. Such explicit procedures are called algorithms or programs. Because an effective notation for the description of programs exhibits considerable syntactic structure, it is called a programming language.”

4.1.1 Types of Programming Languages

Over the decades, many programming languages have been developed, each designed with different goals. A common way to classify them is into low-level and high-level languages.

- **Low-Level Languages:** They provide direct control over hardware but are difficult to write and maintain. Examples are assembly languages.
- **High-Level Languages:** They are closer to human language, easier to write, read, and maintain, and abstract away hardware details. Examples include Python, C/C++, and Rust.

Writing software directly in assembly code is inefficient and error-prone. It is rarely used today, except when low-level control is required, such as in device drivers or operating system development. In contrast, high-level languages are widely used in the software industry. Programs written in high-level languages are eventually translated into machine code, the binary representation of machine instructions, either ahead of time by a compiler or during execution by an interpreter or a just-in-time compiler.

There are also different programming paradigms, depending on how problems are modeled and how computations are expressed in programming languages. For example, *imperative programming* describes a program as a sequence of commands that change the program’s state; *functional programming* emphasizes the evaluation and composition of functions; and *object-oriented programming* organizes programs around objects that encapsulate data and behavior. A programming language may support multiple paradigms. These topics are beyond the scope of this class.

4.1.2 Requirements of Programming Languages

A high-level language should be unambiguous and well-specified.

- **Unambiguous:** For any valid program, there is only one predefined way to interpret it. For example, $a + b * c$ could be interpreted as $(a + b) * c$ or $a + (b * c)$ if the language does not define operator precedence.
- **Well-specified:** The meaning of a program should be sufficiently defined, without leaving unintended behaviors to interpretation. A familiar example comes from the percentage key on traditional calculators. Unlike ordinary mathematical notation, where 50% simply means 0.5, calculators may interpret the percentage operator according to the surrounding context. For example, what should $1 + 50\% + 50\%$ mean? Should it be $1 + 0.5 + 0.5 = 2 = 200\%$, or should each percentage be applied to the current value, resulting in $1 \times 1.5 \times 1.5 = 2.25 = 225\%$? The language must clearly define the semantics of the % operator.

To avoid ambiguity, programming languages generally define their syntax using formal languages, such as regular expressions and context-free grammars. However, syntax alone cannot fully specify the behavior of a program. There may still be some unspecified behaviors. Consider the following expression: $f() + g()$. If a language does not require $f()$ to be evaluated before $g()$, or vice versa, the result could be different when $f()$ and $g()$ have side effects that modify the same data.

In comparison, natural language is especially prone to it. For example:

A programmer's wife asks him to go to the grocery store. She says: "Get a gallon of milk. If they have eggs, get 12." The programmer returns with 12 gallons of milk.

This humorous story shows how natural language can be interpreted in multiple ways. This is one reason why using natural language directly for programming was not practical in the past. With the evolution of large language models, however, writing programs using natural language has become increasingly promising. Still, developers should be aware of two important risks: ambiguity in prompts and under-specification, where a prompt is incomplete or lacks sufficient detail, causing the system to make unintended assumptions.

This course uses Python as the main programming language, because it offers a gentle learning curve while being powerful enough for serious software development.

4.2 Python

Python is an interpreted, high-level programming language, invented by Dutch programmer Guido van Rossum in 1991. It is widely used in areas such as artificial intelligence, data science, and automation, and is famous for its clean, simple syntax. A Python program consists of a sequence of statements that are executed by the Python interpreter. In this section, we will learn the basic components of Python programs. Students are not required to write programs from scratch, but they should be able to read Python programs and make modifications to them.

4.2.1 Values and Variables

A program works with data, and the data used by a program is represented as *values*. For example, 25, 3.14, "Alice", and True are all values in Python. A *variable* is a name that allows us to refer to a value. You can think of a variable as a label attached to a value. For example,

```
x = 25
name = "Alice"
```

Here, `x` is a variable that refers to the value 25, while `name` refers to the value "Alice". The symbol `=` is called the assignment operator. It assigns a value to a variable. A variable can later be assigned a different value. Thus, a variable name does not permanently represent a particular value; it can refer to different values at different points in a program.

```
x = 25
x = 30
```

Values in Python have different *types*. Common types include integers, floating-point numbers, strings, and Boolean values. For example, in the following code, `x` refers to an integer value, `y` refers to a floating-point value, `name` and `age` refer to string values, and `is_student` refers to a Boolean value. Notice that `age = "25"` is a string rather than an integer because the value is enclosed in quotation marks.

```
x = 25           # integer
y = 3.14         # float
name = "Alice"   # string
age = "25"       # string
is_student = True # boolean
```

Unlike some programming languages, Python does not require programmers to explicitly declare the type of a variable. The type of the value is determined at runtime. This is one of the characteristics of Python's dynamic typing. Python also supports optional *type annotations*, which allow programmers to indicate the intended type of a variable. For example,

```
name: str = "Alice"
```

Here, `str` is a type annotation indicating that `name` is intended to refer to a string. Type annotations are optional and are mainly used to make programs easier to understand and analyze.

Another important distinction is between *mutable* and *immutable* values. An immutable value cannot be changed after it is created, while a mutable value can be changed. Common immutable types include integers, floating-point numbers, strings, and Boolean values. Lists and dictionaries, which we will introduce later, are examples of mutable types.

For example, consider the following code. The assignment `x = 20` does not modify the integer value 10. Instead, `x` is reassigned to refer to the value 20. Similarly, `x` can later be reassigned to refer to a value of a different type:

```
x = 10
x = 20
x = "Alice"
```

In other words, a variable does not have to be permanently associated with a particular value or type. Rather, the type belongs to the value to which the variable currently refers.

4.2.2 Naming

A variable name must follow Python's rules for identifiers. It must begin with a letter or an underscore, and the rest of the name can contain letters, digits, or underscores. For example, valid variable names include `abc123`, `_abc`, and `ifabc`, while `123abc` is invalid because it begins with a digit. Certain words, called Python *keywords*, cannot be used as variable names because they have special meanings in the language. Examples include `def`, `for`, and `if`. Notice that `ifabc` is still a valid variable name because it is not a keyword. Python identifiers are case-sensitive. This means that `abc123` and `Abc123` are considered different identifiers.

Python programmers also follow naming conventions to make code easier to read. Common naming conventions include:

- **Variables:** Use *snake_case*, in which words are written in lowercase and separated by underscores. For example, `total_score` and `student_name`.
- **Functions:** Use *snake_case*, just like variable names.
- **Classes:** Use *PascalCase*, in which each word begins with a capital letter. For example, `StudentRecord` and `FudanUniversity`.

4.2.3 Type Checking

In a valid Python program, the types of variables or values must match the requirements of the operators being applied. Otherwise, Python raises an error. As shown in the following code, the first two statements are valid, while the third is invalid because it attempts to add a number to a string, which has no logical meaning. In such cases, Python raises a **TypeError** to prevent the program from executing incorrectly. Besides, Python requires variables must be defined before use. Therefore, statement four is also invalid.

```
a = 5 + 3          # Valid: add two integers
b = "5" + "3"      # Valid: '+' means concatenating two strings; result: "53"
c = 5 + "3"         # Type Error: add an integer to a string is invalid.
x = a + d;          # Invalid: d is not defined.
x = 0.5 + 3         # Type coercion: the result is float 3.5
x = 5 + True        # Type coercion: the result is integer 6
```

In some cases, Python performs type coercion, also called implicit conversion. When two different numeric types are combined in an expression, Python automatically converts the narrower type to the wider type so that the operation can be carried out safely. For example, the fifth statement adds a

floating-point number to an integer: Python first converts the integer to a float, and the result is a float. In the last statement, the boolean value is converted to an integer before the addition. This behavior makes Python both safe and flexible when handling mixed numeric operations.

4.2.4 Control Flow

Control flow statements allow a program to execute different code blocks depending on certain conditions or to repeat code multiple times. Python provides several mechanisms for controlling the flow of execution, including conditional statements and loops.

Conditional Statements

Conditional statements enable a program to make decisions by evaluating boolean expressions.

The most commonly used conditional statements are if statements, which can take three main forms.

- **if statement:** Executes a block of code if a condition is true.

```
if condition:
    print("Code executed if condition is true")
```

- **if-else statement:** Executes one block if a condition is true, another if it is false.

```
if condition:
    print("Code executed if condition is true")
else:
    print("Code executed if condition is false")
```

- **if-elif-else statement:** Handles multiple conditions.

```
if condition1:
    print("Code executed if condition1 is true")
elif condition2:
    print("Code executed if condition2 is true")
else:
    print("Code executed if all conditions are false")
```

The match-case statement in Python provides a way to perform pattern matching, which is useful for handling complex conditions more clearly than multiple **if-elif-else** statements. It allows a variable or expression to be compared against multiple patterns, executing the corresponding block of code for the first matching pattern. A wildcard pattern “_” can be used to define a default case when no other patterns match.

```
match variable:
    case pattern1:
        print("Code executed if the variable matches pattern1")
    case pattern2:
        print("Code executed if the variable matches pattern2")
    case _:
        print("Code executed if the variable does not matches any pattern")
```

Loops

Loops allow a program to repeatedly execute a block of code. The while loop repeats a block of code as long as a given condition is true. It is often used when the number of iterations is not known in advance and depends on some dynamic condition. Before each iteration, Python evaluates the condition; if it is true, the loop body executes, and the condition is checked again. This process continues until the condition becomes false.

```
while condition:
    print("code executed repeatedly if the condition is true")
```

The for loop, on the other hand, repeats a block of code for each item in a sequence of items. You do not need to manually manage loop counters, as Python automatically iterates over each element in the sequence. For loops are particularly convenient when the number of iterations is determined by the length of a collection.

```
for item in items:
    print("code executed for each item")
```

In addition, both while and for loops can include the **break** and **continue** statements within their bodies. The **break** statement immediately exits the loop, allowing the program to skip the remaining iterations. The **continue** statement skips the rest of the current iteration and moves on to the next one. These statements provide finer control over loop execution and make it easier to handle special conditions within loops.

4.2.5 Functions

A function in Python is an abstraction that maps a set of input values to a set of output values. It is a named block of code designed to perform a specific task. Functions can take inputs, called parameters, and optionally produce an output using the return statement. By defining a function once, you can call it multiple times throughout your program, which makes it a reusable unit of work. Functions help organize code, reduce repetition, and make programs easier to read, maintain, and test.

```
def factorial(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
```

Code 4.1: Example function: Factorial in iterative mode

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
```

Code 4.2: Example function: Factorial in recursive mode

Code 4.1 and Code 4.2 both implement the factorial function. Code 4.1 uses an iterative approach based on a **for** loop, repeatedly multiplying values to compute the result. Code 4.2, on the other hand, uses a recursive approach, where the function calls itself to compute the factorial.

4.2.6 Built-in Data Structures

Besides basic data types such as integers and strings, Python provides several other powerful built-in data structures.

Tuple

A tuple is an ordered collection of elements, which can hold heterogeneous data, including numbers, strings, lists, or even other tuples. Tuples are immutable, so once a tuple is created, its elements cannot be modified, added, or removed.

```
t1 = (1, 2, 3)           # Tuple with three integers
t2 = (1, "hello", 3.14)  # Tuple with mixed types
t3 = (1, (2, 3), [4, 5]) # Nested tuple
print(t1[0])             # 1, access the first element of t1
print(t1[0:2])           # 1,2, access the slice of t1
for item in t1:          # Iterate over all elements of the tuple
    print(item)
```

List

A list in Python is similar to a tuple in that it can contain elements of different types, including numbers, strings, or even other lists. Unlike tuples, however, lists are mutable, which means you can add, remove, or modify their elements. This flexibility makes lists a versatile and commonly used data structure in Python programs.

```
a = []                  # Empty list
b = [1, 2, 3]           # List with elements
c = [1, "hello", 3.14]  # List with mixed types
print(b[0])             # access the first element of b
b.append(4)             # Add at the end
b.insert(2, 100)         # Insert 100 at index 1
b.remove(2)             # Remove first occurrence of 2
for n in b:             # Iterate over all elements of the List
    print(n)
```

Dictionary

A dictionary in Python is a built-in data structure that stores data as key-value pairs. Each key in a dictionary must be unique, while values can be of any type and may be duplicated. Dictionaries are very efficient for lookups, allowing you to quickly retrieve or modify a value using its key. We will explain the details of the implementation in the next lecture.

```
students = {
    "Alice": 20,
    "Bob": 21,
    "Charlie": 19
}
print(students["Bob"]) # Output: 21
```

4.2.7 Class

Python also supports classes, which are a programming abstraction that defines a new type of object. A class specifies the attributes (data) and methods (behavior) that its instances will have. Similar to a function acting as a reusable unit of work, a class is defined once, and you can create multiple instances. Each instance shares the same structure and behavior defined by the class but can hold different data, making classes a powerful tool for organizing and reusing code.

```
import math # Import the math library

class Ellipse: # define a class named Ellipse
    def __init__(self, f1, f2, radius_sum): # Constructor
        self.f1 = f1
        self.f2 = f2
        self.radius_sum = radius_sum
        dist_foci = math.dist(f1, f2)
        if radius_sum <= dist_foci:
            raise ValueError("Invalid ellipse")

    def contains_point(self, x, y): # Is (x,y) inside the ellipse?
        dist_to_f1 = math.dist((x, y), self.f1)
        dist_to_f2 = math.dist((x, y), self.f2)
        return dist_to_f1 + dist_to_f2 <= self.radius_sum

ellipse = Ellipse((0, 0), (4, 0), 6)
print(e.contains_point(3, 0))
```

Code 4.3: Class in Python

Code 4.3 defines a class `Ellipse`, which models an ellipse using its two foci and the sum of distances from any point on the ellipse to the foci. It contains two methods (functions of a class): the constructor `__init__` (default name) and the method `contains_point`. The constructor initializes the ellipse by storing the foci and radius sum, and it checks that the radius sum is greater than the distance between the foci to ensure a valid ellipse. The method `contains_point` returns `True` if the point (x, y) lies inside or on the ellipse, and `False` otherwise. The use case demonstrates creating an ellipse with foci at $(0, 0)$ and $(4, 0)$ and a radius sum of 6. It then tests whether the point $(3, 0)$ is inside the ellipse.

4.3 Agentic Coding with Python

Vibe coding is a recent concept introduced by Andrej Karpathy. It is an approach in which developers use LLMs to generate code from natural language prompts, rather than writing it manually. Developers can run the generated code, observe the results, and iteratively prompt the LLM to improve it. Agentic coding takes this idea a step further. An AI coding agent cannot only generate code, but also run the code, inspect errors, modify files, and iteratively improve the program. In this way, the agent can perform multiple steps to complete a programming task.

Since LLMs and AI coding agents are already powerful programming tools, this course does not expect students to write Python programs from scratch. Instead, students are expected to:

- **Describe the task:** Clearly explain what the program should do.

- **Guide the agent:** Provide prompts and additional instructions when necessary.
- **Read the code:** Understand the generated code and how it works.
- **Test the program:** Run the generated program and check whether it works as expected.
- **Evaluate and improve:** Identify errors or problems and ask the agent to improve the program.

The goal is not simply to generate code with AI, but to learn how to effectively use AI agents to develop, test, and improve programs.

Exercise

- 1) Consider the following two functions. What do they compute? *i.e.*, given any input n , what does the output mean? What is the difference between the two functions?

```
import math

def is_prime_v1(n):
    if n < 2:
        return False

    for i in range(2, n):
        if n % i == 0:
            return False

    return True

def is_prime_v2(n):
    if n < 2:
        return False

    limit = int(math.sqrt(n))

    for i in range(2, limit + 1):
        if n % i == 0:
            return False

    return True
```

- 2) Prompt an AI coding agent (*e.g.*, OpenCode + DeepSeek) to write a simple game, such as Snake or Tank Battle. Examine the generated code and run the game. Then, use further prompts to request specific modifications and check whether the agent correctly implements the requested changes.